

Farhan Akbor Khan

Assignment - 02

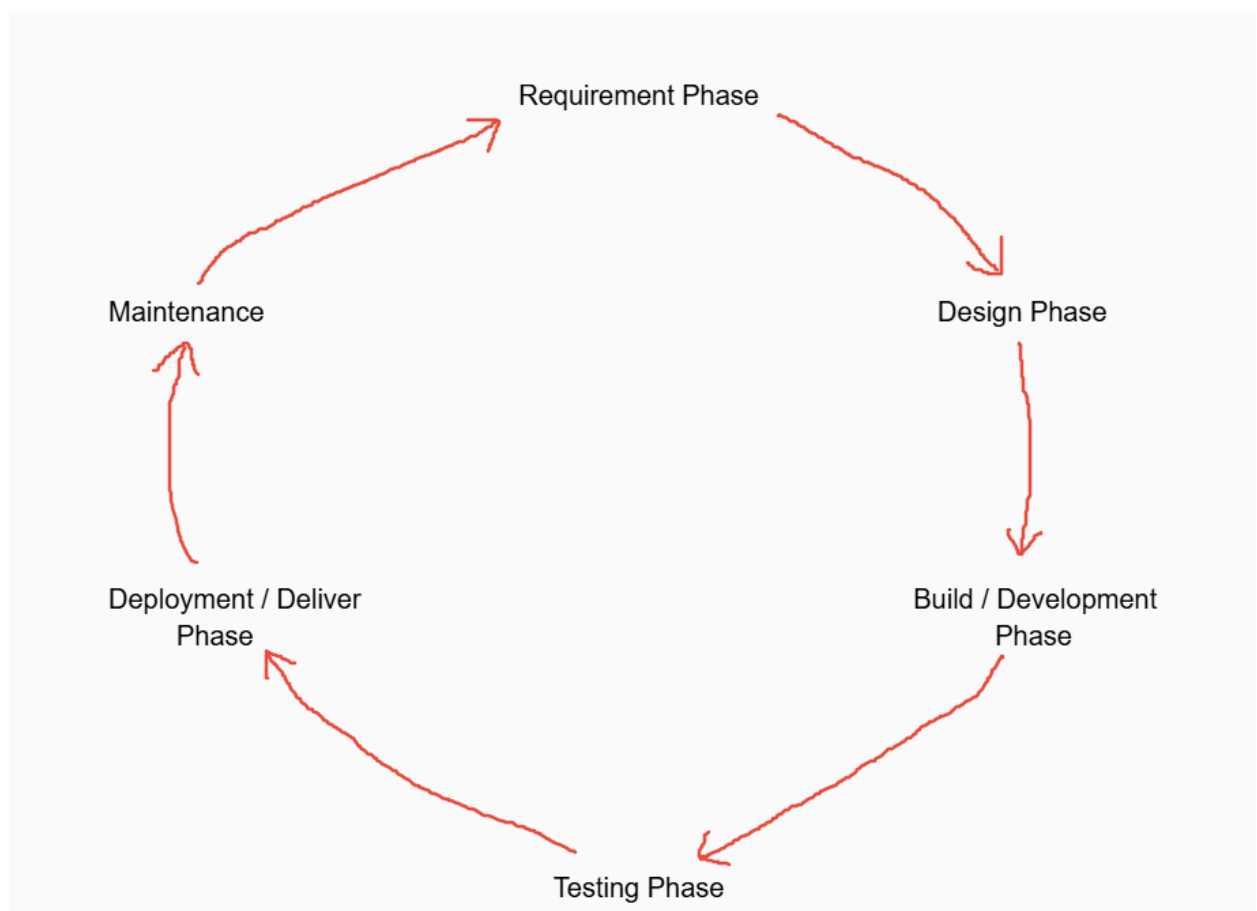
Module-02

Software Development Life Cycle (SDLC) in the Age of AI

Q1. Draw the SDLC lifecycle diagram and briefly explain the workflow.

Answer:

SDLC Lifecycle Diagram



1. **Requirement Analysis:** The customer's requirements are collected and analyzed. The project's scope, feasibility, cost and resources are determined.
2. **Design Phase:** The system architecture, database, user interface, modules and technologies are designed based on the requirements.
3. **Development Phase:** Developers convert the design into actual source code and build the different system modules.
4. **Testing:** The developed software is tested to identify and fix bugs and to ensure that it meets the specified requirements.
5. **Deployment:** After successful testing, the software is released and installed in the production environment for end users.
6. **Maintenance:** The software is monitored and updated after deployment. Bugs are fixed, security issues are addressed and new features may be added.

In short:

Requirements → Design → Coding / Development → Testing → Deployment → Maintenance → Feedback → New Requirements.

Q2. Practical AI Exercise

Choose one AI tool (ChatGPT, Claude, GitHub Copilot, Cursor AI, or Gemini) and perform the following:

Generate five user stories for an E-commerce application.

Convert one user story into at least five acceptance criteria.

Ask the AI to create a Sprint Backlog.

Review the AI-generated output and improve it manually.

Explain:

What the AI did well.

What corrections did you make?

Why human review is still necessary.

Attach screenshots of your AI interaction.

Generate five user stories for an E-commerce application.

User Story 1 (Product Search & Filtering):

As an online shopper, I want to filter products by price, brand, and customer ratings so that I can quickly find items that meet my budget and quality preferences.

User Story 2 (Cart Management):

As a returning customer, I want to save items to a wishlist so that I can purchase them later when they go on sale.

User Story 3 (Checkout & Payment):

As a mobile buyer, I want to pay using Apple Pay or Google Pay so that I can complete my order rapidly without re-entering card details.

User Story 4 (Order Tracking):

As a customer, I want to view real-time tracking updates for my dispatched package so that I know exactly when it will arrive at my address.

User Story 5 (User Reviews):

As a verified buyer, I want to upload photos alongside my text review so that I can share authentic product feedback with other shoppers.

Convert User Story 3 into at least 5 detailed acceptance criteria

Criterion 1 (Availability):

Given I am on the Checkout screen on a supported mobile device, When I select express payment options, then I should see Apple Pay (on iOS) or Google Pay (on Android) presented as available payment methods.

Criterion 2 (Successful Authorization):

Given I choose Apple Pay / Google Pay, When I authenticate using biometrics (Face ID/Touch

ID/Fingerprint), *then* the transaction should be processed immediately and redirect me to the Order Confirmation screen.

Criterion 3 (Address Auto-population):

Given I select biometric payment, *When* the payment sheet opens, *Then* my default shipping address stored in Apple/Google Wallet should automatically populate the shipping address field.

Criterion 4 (Payment Failure Handling):

Given my linked card has insufficient funds or fails authorization, *When* the transaction is attempted, *Then* the app displays a clear error message: "Payment declined. Please select an alternative payment method" and keep my cart intact.

Criterion 5 (Order Receipt Generation):

Given a successful payment, *When* the transaction completes, *Then* an automated order receipt email must be sent to the email address tied to the account within 60 seconds.

Create a 2-week Sprint Backlog for implementing User Story 3 and its acceptance criteria.

Task ID	Task Description	Assigned Role	Est. Effort (Story Points / Hours)
TSK-101	Integrate Stripe/Adyen SDK for Apple Pay & Google Pay	Backend Developer	5 Points (12 hrs)
TSK-102	Design express payment buttons on mobile UI	UI/UX Designer	2 Points (4 hrs)
TSK-103	Implement iOS Native SDK triggers for Face ID / Touch ID	Mobile Dev (iOS)	3 Points (8 hrs)
TSK-104	Implement Android Google Pay API integration	Mobile Dev (Android)	3 Points (8 hrs)

TSK-1 05	Write automated E2E payment tests	QA Engineer	3 Points (6 hrs)
TSK-1 06	Create confirmation email triggers	Backend Developer	2 Points (4 hrs)

1. What the AI Did Well

- **Speed & Structure:** Gemini generated standardized, syntax-accurate user stories and BDD-style acceptance criteria in seconds.
- **Format Adherence:** It adhered strictly to requested Agile formats (As a... I want to... So that... and Given-When-Then).
- **Baseline Coverage:** It accurately identified core happy-path functional requirements (UI buttons, payment triggers, basic confirmation).

2. What Corrections Were Made Manually?

- **Added Edge Cases & Real-Time Logistics:** Introduced address recalculation logic (taxes/shipping updates dynamically when altered inside the wallet sheet).
- **Enforced Security & Compliance Standards:** Added PCI-DSS compliance, tokenization requirements and security review tasks overlooked by the LLM.
- **Expanded Quality Assurance Scope:** Replaced simple "E2E testing" with specific edge-case scenarios (network failure, biometric cancelation, webhook retries).

3. Why Human Review is Still Necessary

- **Domain & Business Logic Context:** AI models lack contextual awareness of legacy backend architectures, third-party API quirks, tax rules, and legal compliance.
- **Nuance & Edge Case Blindness:** LLMs tend to generate "happy path" scenarios (where everything works perfectly). Human product owners and engineers are essential for anticipating system failures, security risks and user errors.

- **Agile Estimation Reality:** AI cannot gauge a team's actual velocity, skill distribution or historical technical debt. Human engineering leads are required to accurately estimate task complexity.

Task ID	Work Item & Technical Description	Assigned Role	Complexity (Story Points)
TSK-101	Backend: Set up Stripe PaymentIntent API & Webhooks for tokenized Apple/Google Pay processing.	Backend Dev	5
TSK-102	Mobile UI: Integrate Apple/Google Pay native buttons adhering to Apple Human Interface & Material Design guidelines.	iOS / Android Mobile Devs	3
TSK-103	Middleware: Implement shipping/tax recalculation hook during wallet sheet callback events.	Backend Dev	3
TSK-104	Security & Compliance: Conduct PCI-DSS tokenization audit and data sanitization verification.	SecOps / Dev	2
TSK-105	QA / Testing: Write automated sandbox E2E tests for edge	QA Engineer	3

	cases (declined cards, biometric cancel, timeout).		
TSK-106	DevOps: Configure webhook retries & dead-letter queue for asynchronous order receipts.	DevOps Eng	2
Total	Total Estimated Sprint Complexity	Cross-Functional Team	18 Story Points

◆ Upgrade

Generate five user stories for an E-commerce application.



Convert this user story into at least five acceptance criteria.

Order Tracking



create a Sprint Backlog